

Chapter 14

Contents

Chapters

14. Interfacing DC, Stepper & Servo Motors

14.1 Interfacing of DC Motor

14.2 Interfacing of Stepper Motor

14.3 Interfacing of Servo Motor

14.1 Interfacing of DC Motor

The PIC18 devices or any microprocessor or a microcontroller typically has port pins that has current limits of around milli amperes, with some in the micro ampere range. To drive any high current devices external drivers or power supplies must be used. Hence microcontrollers are typically interfaced through Relays.

Relays are devices that act as an electrically operated switch. These are typically electromechanical or solid-state relays. Electromechanical relays operate based on current passage to energize or de-energize a coil, to open or close a switch. Solid state relays make use of semiconductors such as transistors or MOSFETS to operate as a switch – open or close without any moving parts. Other variations include use of LED's & Opto-couplers. The L293, L298 drivers presented later for motor interfacing makes use of solid-state approach.

Similarly, when interfacing motors and other equipment's that act as an inductive load, and for isolation of low power devices like microcontrollers isolators must be used. Isolators are devices that separate, protect or decouple two or more parts of a system.

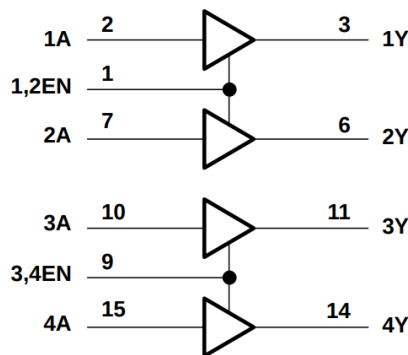


Fig 14.1 Internal Logic Diagram of L293/293D Half Bridge Driver

Source: L293, L293D Quadruple Half-H Drivers; SLRS008B

September 1986 – Revised June 2002.

Table 14.1: Input Selection for direction control of motor

Inputs		Function
$V_{EN} = H$	IN1 = H ; IN2 = L	Forward
	IN1 = L ; IN2 = H	Reverse
Other Combinations		Fast Motor Stop

The internal logic diagram of L293D quadruple Half Bridge Driver is shown in Fig 14.1. It has 4 Half Bridges with 4 Enable signals, 4 inputs and 4 outputs. Additional details may be obtained from the L293D data sheet. L293D provide protection from voltage spikes using its built-in flyback (clamping) diodes. The interfacing of a 12V DC motor using a L293D

Driver is presented in Fig 14.2. A sample interfacing of multiple DC motors using L293 is presented in Fig 14.3.

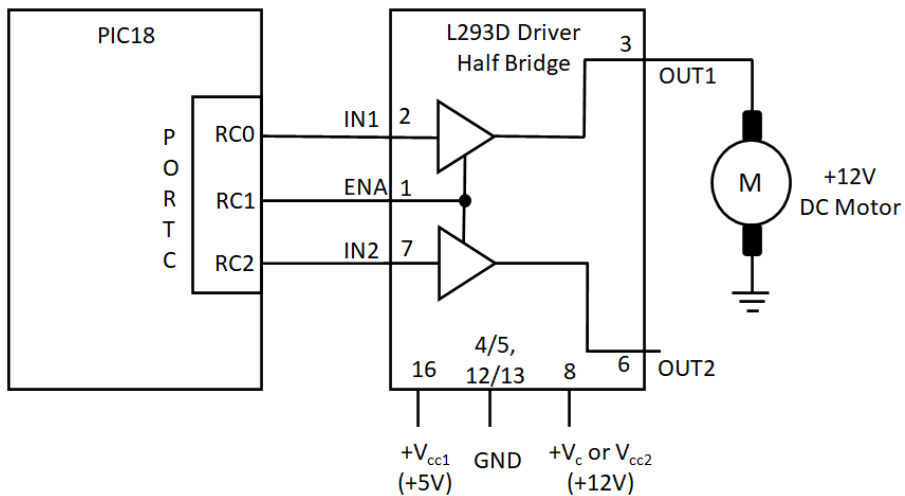


Fig 14.2 Interfacing DC Motor (Uni-Direction) using L293 Quadruple Half Bridge Driver

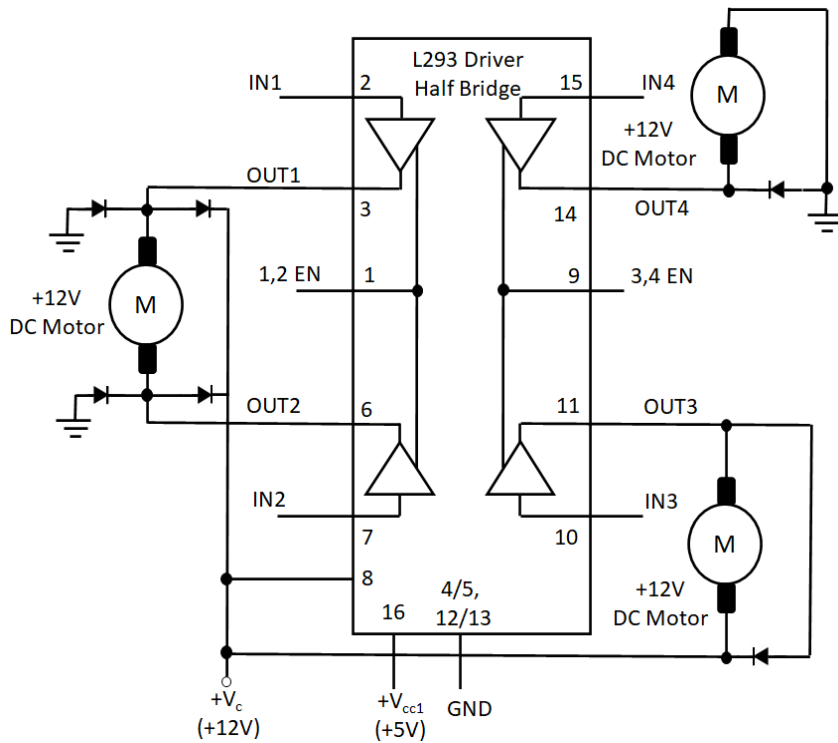


Fig 14.3 Interfacing multiple DC Motors Application using L293

The internal logic diagram of L298 dual high voltage high current Full Bridge Driver is shown in Fig 14.4. It has 2 Full Bridges with 2 Enable signals, 4 inputs and 4 outputs. Additional details may be obtained from the L298 data sheet. The full bridge circuit provides up to 2A current per channel compared to 600mA in a L293D driver, however 298N requires external diodes for inductive loads. The L298 also has heat sink and higher operating voltage up to 46V while L293D is operable up to 36V. It offers a full bridge feature compared to half bridge of L293D. The interfacing of a 12V DC motor using L298 Driver is presented in Fig 14.5.

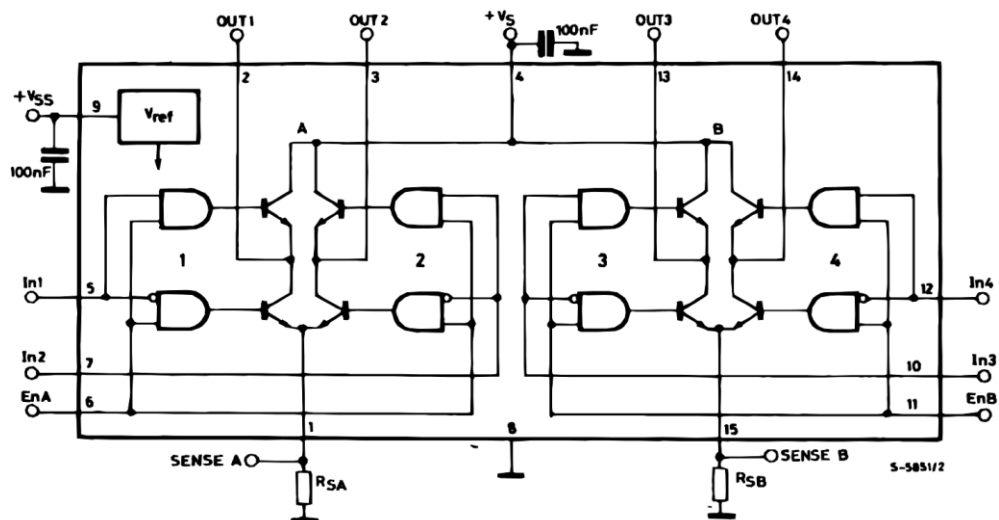


Fig 14.4 Internal Logic Diagram of L298 Dual Full Bridge Driver

Source: L298 Dual Full-Bridge Driver; ST Microelectronics

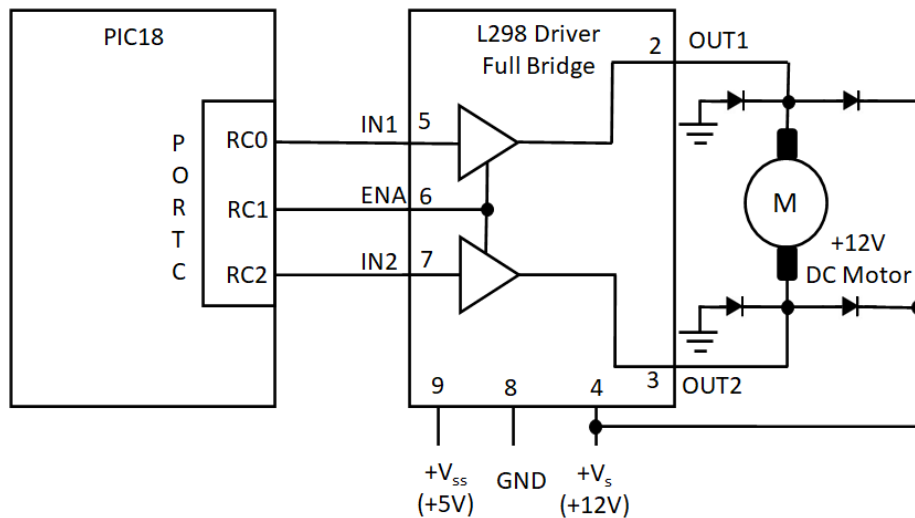


Fig 14.5 Interfacing DC Motor (Bi-Direction) using L298 Full Bridge Drivers

A second motor may be connected to output terminals OUT3 (13), OUT4 (14) and can be controlled through IN3 (10), IN4 (12), ENB (11). Table 14.2 provides input selection for direction control of motor. A sample circuit shown in Fig 14.6 demonstrates the interfacing of a high current dc motor where total current generated from two full bridge drivers is used to drive the same motor.

Table 14.2: Input Selection for direction control of motor

Inputs		Function
$V_{EN} = H$	IN1 = H ; IN2 = L	Forward
	IN1 = L ; IN2 = H	Reverse
	IN1 = IN2	Fast Motor Stop
$V_{EN} = L$	IN1 = X ; IN2 = X	Free Running Motor Stop

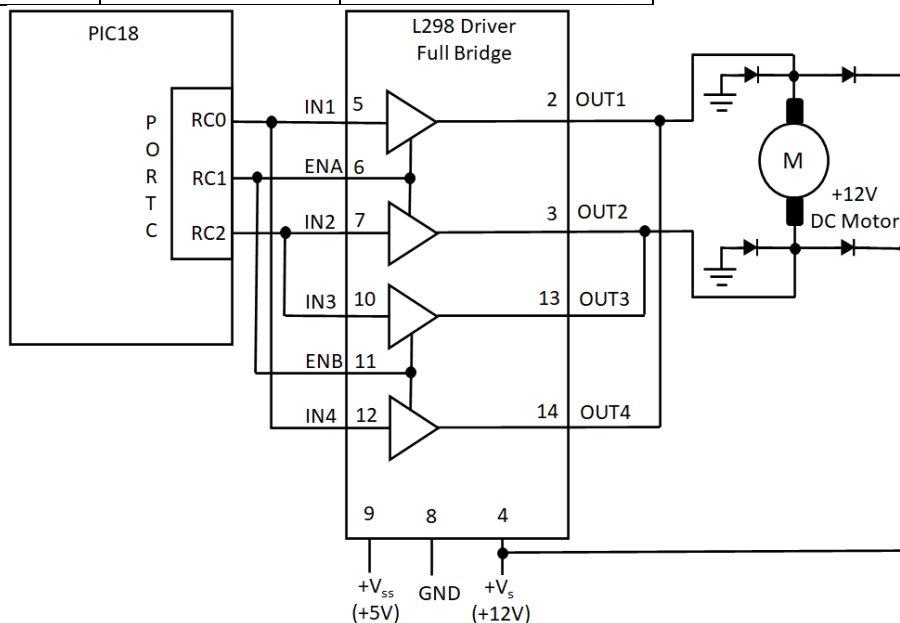


Fig 14.6 Interfacing High current DC Motor using L298's two Full Bridge Drivers

Program 14.1: Write an ALP to turn ON the DC motor of a conveyer belt for shifting the packaged items to another end of assembly line when switch is pressed and turn it off when deselected.

Sol: Assume that the DC motor is driven by PIC18 PORTC pin 0, enabled through PORTC pin1 and a switch is connected at PORTB pin 5 as shown in Fig P14.1.

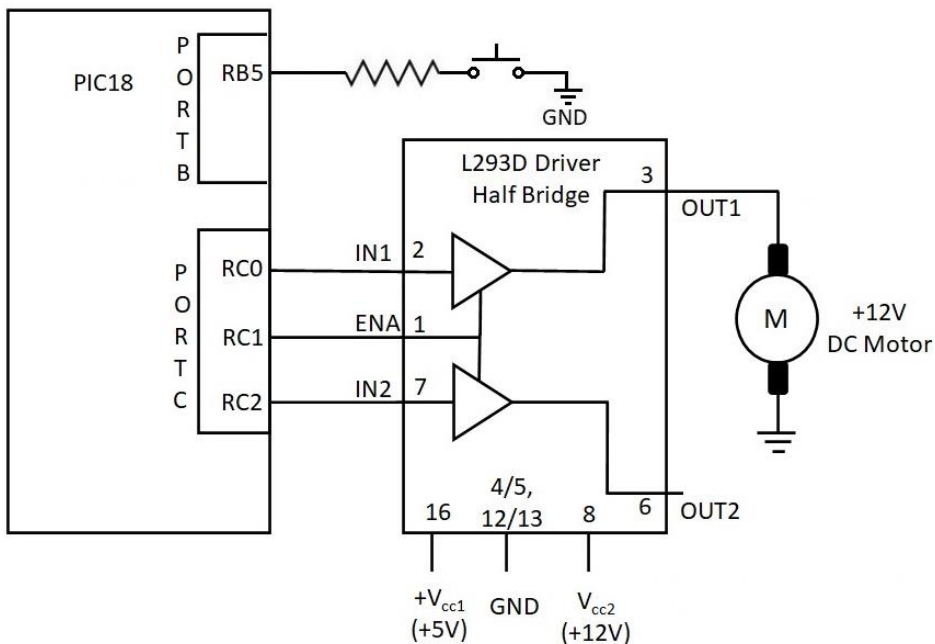


Fig P14.1 Conveyor Belt Motor Interfacing using L293D driver

	#INCLUDE <P18F458.INC>		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x0100	; Set Origin 0x0100 of Main Program
START	MOVLW	B'00100000'	; WREG = 0x20
	MOVWF	TRISB	; Configure PORTB pin 5 as input. ; TRISB = 0x20
	BCF	INTCON2,RBPU	; Enable the weak pull up for PORTB ; Internal weak Pull-up is enabled,
	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISC	; Configure PORTC as output. TRISC = 0x20
HERE	MOVF	PORTB,0	; Read PORTB to WREG, WREG = PORTB
	ANDLW	0x20	; Extract status of Port B pin 5
	BZ	IL1	; If RB5 is 0, go to IL1
	BCF	PORTC,0,0	; RB5 is 1, Button Not Pressed ; So, turn OFF the Motor, Disable IN1 at RC0
	BCF	PORTC,1,0	; Driver disable ENA at RC1
	BRA	IL2	; Go to IL2
IL1	BSF	PORTC,0,0	; RB5 is 0, Button Pressed ; So, turn ON the Motor, enable IN1 at RC0
	BSF	PORTC,1,0	; Driver enable ENA at RC1
IL2	BRA	HERE	

	END		
--	-----	--	--

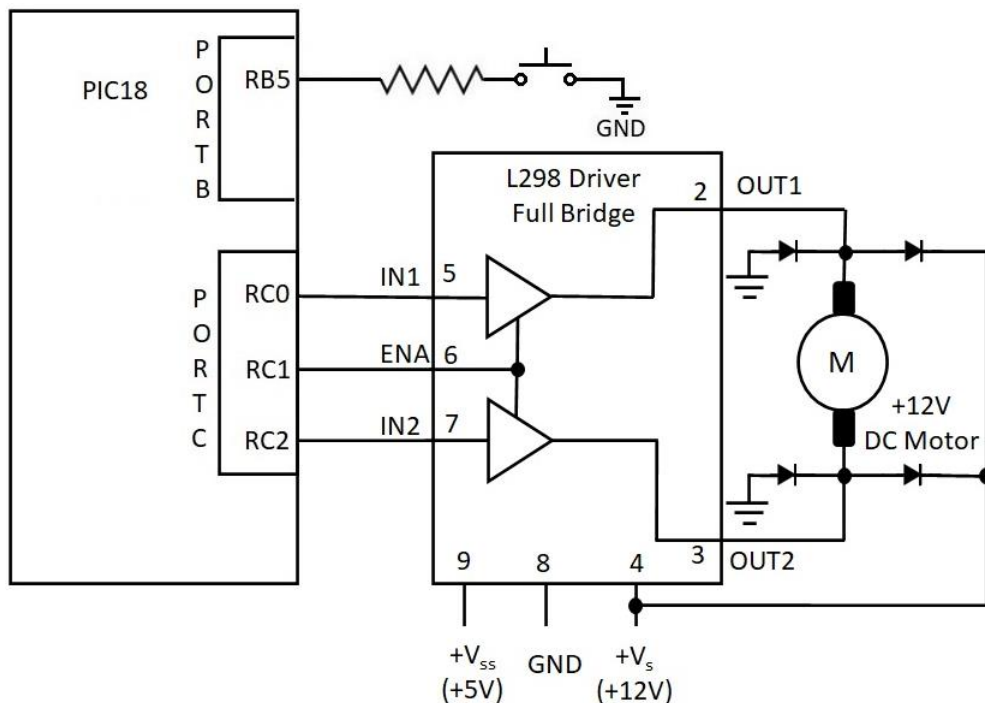


Fig P14.2 Garage door interfacing using L298 driver

Program P14.2: Write an ALP to rotate DC motor anti clockwise for DLY seconds to open a garage door. After a delay of 2 DLY seconds, close the garage door by rotating clockwise for DLY seconds. Assume that the DC motor is driven through PIC18 PORTC based on switch at PORTB pin 5.

Sol: Here the program is presented for a conceptual delay of DLY seconds. The DLY may be representing 10 seconds or 5 milli seconds etc. The concept of creating delays is discussed in Timers of chapter 9. Assume that the DC motor is driven by PIC18 PORTC pin 0 and pin 2 in opposite logic level, enabled through PORTC pin1 and a switch is connected at PORTB pin 5 as shown in Fig P14.2.

	#INCLUDE <P18F458.INC>		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x50	; Set Origin 0x0050 of DLY Module
DLY	MOVLW	0x03	; WREG = 0x03, The required loop value ; depends on motor & oscillator frequency.
	MOVWF	0x11,0	; [0x11] = WREG = 0x03
L2:	MOVLW	0x04	; WREG = 0x04, The required loop value

			; depends on motor & oscillator frequency.
	MOVWF	0x10,0	; [0x10] = WREG = 0x04
L1:	DECf	0x10, F	; Decrement File Register 0x10, ; [0x10] = [0x10] - 1
	BNZ	L1	; If not zero Branch to L1.
	DECf	0x11, F	; Decrement File Register 0x11, ; [0x11] = [0x11] - 1
	BNZ	L2	; If not zero Branch to L2.
	RETURN		; Return from Delay module
	ORG	0x0100	; Set Origin 0x0100 of Main Program
START	MOVLW	B'00100000'	; WREG = 0x20
	MOVWF	TRISB	; Configure PORTB pin 5 as input. ; TRISB = 0x20
	BCF	INTCON2,RBPU	; Enable the weak pull up for PORTB ; Internal weak Pull-up is enabled,
	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISC	; Configure PORTC as output. TRISC = 0x00
HERE	MOVf	PORTB,0	; Read PORTB to WREG, WREG = PORTB
	ANDLW	0x20	; Extract status of Port B pin 5
	BNZ	IL1	; If RB5 is 1, go to IL1
	MOVLW	0x03	; WREG = 0x03
	MOVWF	PORTC,0	; PORTC = 0x03; Activate IN1 & ENA to ; rotate motor in CW direction to open ; garage door.
	CALL	DLY	; Call Delay module; Create DLY seconds ; i.e. Motor powered ON with forward ; direction for DLY seconds.
	MOVLW	0x00	; WREG = 0x00
	MOVWF	PORTC,0	; PORTC = 0x00; Switch OFF motor.
	CALL	DLY	; Call Delay module; Keep Garage door open
	CALL	DLY	; Call Delay module for 2 DLY seconds.
	MOVLW	0x06	; WREG = 0x06
	MOVWF	PORTC,0	; PORTC = 0x06; Activate IN2 & ENA to ; rotate motor in CCW direction to close ; garage door.
	CALL	DLY	; Call Delay module; Create DLY seconds ; i.e. Motor powered ON with reverse ; direction for DLY seconds.
IL1	MOVLW	0x00	; WREG = 0x00
	MOVWF	PORTC,0	; PORTC = 0x00; Switch OFF motor.
	BRA	HERE	
	END		

Program P14.3: Write an ALP to measure the speed of the motor.

Sol: Assume that the DC motor is driven through PIC18 PORTC<5:7>. The rotation of the rotor is measured using a slotted disk and picked up by a proximity sensor/switch which is connected at PORTC pin 0. The speed is sensed by counting the number of pulses based on the detection of slots in the slotted disk.

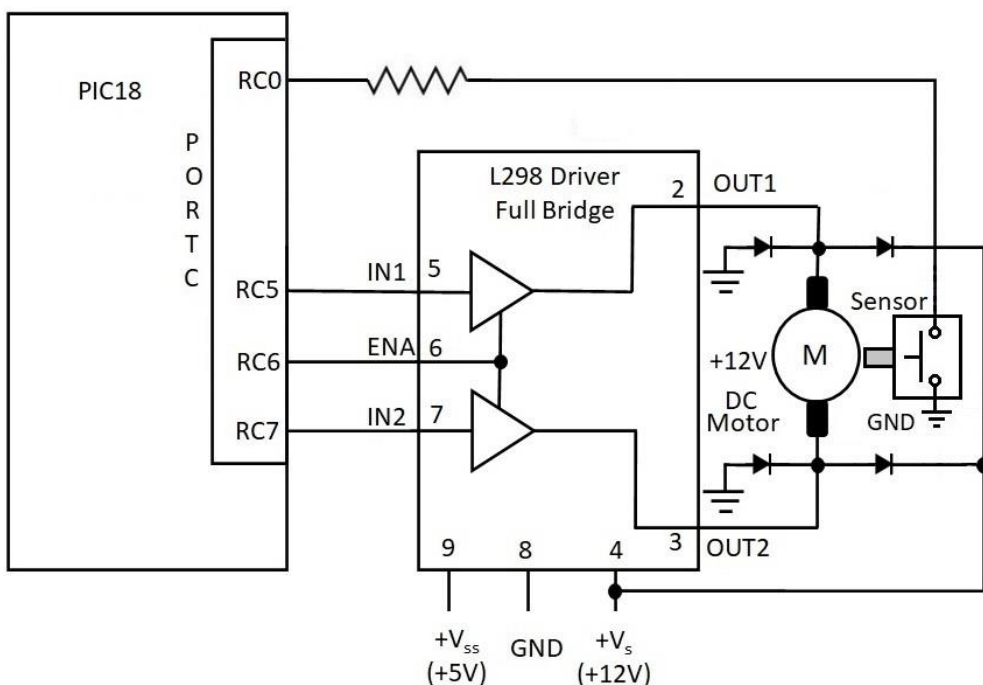


Fig P14.3 Motor & Speed sensor interfacing

For a rotor disk that is single slotted, one pulse is generated per rotation and the number of pulses counted in a second is used to determine the speed. The timing for 1 second may be achieved using a hardware timer and pulses counted using a counter as shown in timer section in counter Program 9.32. Here RC0 is used as input to the timer/counter1 for counting pulses of proximity sensor switch and data may be used to display the speed/number of rotations/pulses per second by sending the value to PORTB and PORTD where timer 3 is used for timing for 1 sec using 25MHz crystal oscillator. The variation w.r.t program 9.32 is de-highlighted here which is for configuring enables pins as output and driving them high for running the motor.

	#INCLUDE <P18F458.INC> ; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.

	ORG	0x100	; Set Origin 0x0100 for Main Program
START	CLRF	TRISB,0	; Set PortB as Output, TRISB=0
-	CLRF	TRISD,0	; Set PortC as Output, TRISD=0
START	MOVLW	B'0000001'	; WREG = 0x01
	MOVWF	TRISC	; Configure PORTC pin 0 as input, all other ; pins of PORTC as output. TRISC = 0x01
	BSF	PORTC,5,0	; Activate IN1 of driver
	BSF	PORTC,6,0	; Activate ENA of driver
	MOVLW	0x82	; WREG = 0x82
	MOVWF	T1CON	; T1CON = WREG = 0x82, 16-Bit Read/Write ; Mode, T1OSC OFF, Ext Clock, ; Synchronization, without Prescale
	MOVLW	0x82	; WREG = 0x82
	MOVWF	T3CON	; T3CON = WREG = 0x82, without Prescale, ; postscale
	LFSR	0,0x10	; FSR0 = 0x10
HERE	MOVLW	0x64	; 100 Loop count to give 1sec.
	MOVWF	0x00	; File Reg [0x00] = 0x64 = 100d
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1H	; TMR1H = WREG = 0x00
	MOVLW	0x00	; WREG = 0x00
	MOVWF	TMR1L	; TMR1L = WREG = 0x00
	CALL	MTR1	; Call MTR1 module
	MOVFF	TMR1H,PORTB	; Move TMR1H value to PORTB
	MOVFF	TMR1L,PORTD	; Move TMR1L value to PORTD
	MOVF	TMR1L,0,0	; Get 16-Bit timer value, WREG = TMR1L, ; TMR1H = TMR1 High Byte
	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0+1
	MOVF	TMR1H,0,0	; WREG = TMR1H
	MOVWF	POSTINC0,0	; [FSR0] = WREG, FSR0 = FSR0+1
	BRA	HERE	; Branch to Here
	ORG	0x200	; Set Origin 0x0200 for MTR1 module
MTR1	BCF	PIR1,TMR1IF	; Clear Timer 1 Flag
	BSF	T1CON,TMR1ON	; Start the Timer/Counter1
L1	BCF	T3CON,TMR3ON	; Stop the Timer3
	MOVLW	0x0B	; WREG = 0x0B
	MOVWF	TMR3H	; TMR3H = 0x0B
	MOVLW	0xDC	; WREG = 0xDC
	MOVWF	TMR3L	; TMR3L = 0xDC
	BCF	PIR2,TMR3IF	; Clear Timer 3 Flag
	BSF	T3CON,TMR3ON	; Start the Timer3
AGAIN	BTFS	PIR2,TMR3IF	; Check if TMR3IF is set. Skip next inst if set.

	BRA	AGAIN	; Branch to Again, when TMR3IF is not set.
	DECFSZ	0x00,1,0	; Decrement File Reg [0x00], skip next inst if ; value is 0.
	BRA	L1	; Branch to L1 to do timer3 100 times.
	RETURN		; Return from MTR1 Module
	END		

Program P14.4: Write an ALP to determine the direction of a rotor using an encoder.

Sol: Assume that the Encoder is connected to PIC18 PORTB<0:1>. The encoder provides two outputs A and B. The direction of rotation of the rotor could be determined from the output signals A and B as on signal would lead while the other lags.

If the falling edge of the output A – i.e clock signal coincides with a logic high on the output B then the direction is clockwise, and if the falling edge of output A coincides with a logic 0 on the output B then the direction is considered to be counter clockwise.

Information on external interrupts – Repeated from Ch7 – Ports & Interrupts:

PIC18F458 has 3 external interrupts INT0, INT1 & INT2 on Port B<0:2> and can be configured based on negative edge or a positive edge. The INTEDGx bit in INTCON2 register can be used to configure the INTx external interrupts to be triggered based on negative/falling (high to low) edge or a positive/rising (low to high) edge. When the INTEDGx bit is set to 1 rising edge is selected and clearing to 0 selects a falling edge.

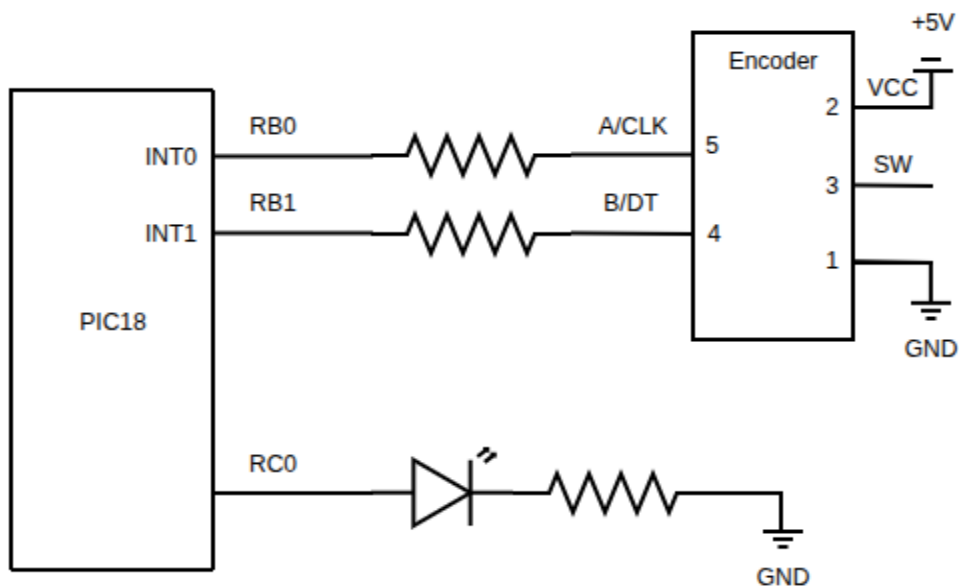


Fig P14.4 Encoder based direction measurement

An interrupt flag INTxIF is generated when the corresponding edge is detected at the respective RBx/INTx pin. The corresponding INTxIF interrupts are enabled or disabled through the INTxIE bit. Setting this bit to 1 enables it and clearing to 0 disables the interrupt. Similar to any other interrupt flag in PIC18, the INTxIF bit must be cleared by user's software (i.e. in ISR if interrupts are enabled or in main program if it is based on polling) before re-enabling the interrupt.

The Port B<0> must be configured to generate an interrupt on falling edge. When the execution is transferred to ISR, the Port B<1> must be checked to see if it has a logic high or a low to determine the direction of the rotor. The program 7.8 of Ch.7 could be modified to achieve this requirement.

	#INCLUDE <P18F458.INC>; CAN BE OTHER PROCESSOR		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x08	; Set Origin 0x08 for High Priority & ; 0x18 for Low Priority Interrupt
	BTFSS	INTCON, INTOIF	; Check if interrupt is due to INTOIF, ; if its due to INTOIF go to XO_ISR
	RETFIE		; If INTOIF not set, return from interrupt.
	GOTO	XO_ISR	; Go to XO_ISR
	ORG	0x100	; Set Origin 0x100 for RB ISR
XO_ISR	BTFSS	PORTB, 1	; Check if PortB<1> is set, ; if its set go to CW
	GOTO	IL1	; If not high, it means CCW, go to IL1
	MOVLW	0xFF	; Its high, it means CW, ; Storing 0xFF for file reg 0x10
	BSF	PORTC,0	; Set LED at RC0 ON, CW
	GOTO	IL2	; Go to IL2
IL1	MOVLW	0x00	; Since CCW, Storing 0x00 for reg 0x10
	BCF	PORTC,0	; Clear LED at RC0 OFF, CCW
IL2	MOVWF	0x10,0	; Set file reg 0x10 to appropriate value
	BCF	INTCON, INTOIF	; Clear INTOIF flag
	RETFIE		; Return from fast Interrupt Enable
	ORG	0x200	; Set Origin 0x200 for Main program
START	BSF	TRISB,0	; TRISB.0 = 1, Make PORTB Pin 0 as Input
	BSF	TRISB,1	; TRISB.1 = 1, Make PORTB Pin 1 as Input
	BCF	TRISC,0	; TRISC.0 = 0, Make PORTC pin 0 as O/P
	BCF	INTCON2, RBPU	; Enable the weak pull up for PORTB
	BCF	INTCON, INTOIF	; Clear INTOIF flag
	BCF	INTCON2, INTEDG0	; Set Falling (High to Low) edge detection
	BSF	GIE	; Enable Global Interrupts

	BSF	INTCON, INTOIE	; Enable Ext Int0
M1	NOP		; No Operation, For Alternate Logic
	BRA	M1	; Branch to M1 to repeat, Wait on INTO
	END		

Program P14.5: Write an ALP to determine the position of a rotating motor using an encoder.

In an incremental encoder, the position of the motor could be determined by counting the number of pulses. It normally transmits these pulses in a sequence on a single line based on the rotation. The counter example presented in Ch. 9 – Timers presents the example of counting the number of pulses in Program 9.29.

An absolute encoder directly gives the position of the motor. It typically has multiple signal lines or provides a sequence of digital bit values serially on a single line using a serial protocol. An 8-bit encoder with 8 signal lines could be connected to an 8-bit port and the port value may be read to determine the position of the rotor. An encoder that transmits data serially using a serial protocol such as USART uses an USART Receive interface as presented in Ch.10 in Program 10.6. The received value represents the position of the encoder.

Program P14.6: Write an ALP to control the speed of the motor using PWM.

A sample program of open loop speed control is presented in program P13.8. The PWM can be modified using CCP or ECCP. A closed loop control system requires a combination of P14.3 and P13.8 with a dynamic variation of duty cycle based on the measured speed and may incorporate PID or any other controller logic.

14.2 Interfacing of Stepper Motors

14.2.1 Introduction to Stepper Motors

The common DC motors rotate continuously when they are powered by a fixed DC voltage. Hence, they are applicable in cases where continuous or multiple rotations are desired, and they are used in closed loop configurations. A common DC motor cannot hold its stop position when the input power is turned off. The stop position may change due to applied load, gravity etc. which generates a back emf voltage.

Various applications exist in which precise repeatable steps are desired. The printers, scanners, camera lenses CD/DVD drives, intelligent lighting, medical imaging, banking systems, gaming machines, Industrial machines for CNC, textile, packaging & automation etc. are some applications for stepper motors. A stepper motor is typically used in open loop configurations.

The stepper motors divide a rotation into a fixed number of discrete angular steps. They rotate by making a discrete step change in position and hence are called as stepping motors. The stepper motors are brushless synchronous electrical motors that convert digital pulses generated by the computers – microprocessors/microcontrollers into mechanical rotation of the shaft.

The advantage of stepper motors is that they make the precise step change for each winding excitation, generate good torque & acceleration with better response, operable for long periods, hold their position, can be precisely controlled and is repeatable.

14.2.1.1 Construction Types

Based on their construction the stepper motors are classified as

- Permanent Magnet type
- Variable Reluctance type
- Hybrid type

Permanent Magnet Type – PM Type

The PM type stepper motors consist of wire wound stator poles surrounding a permanent magnet rotor pole. The direction of rotation depends on the polarity of applied status current. This construction provides good torque and holds its position but has lower resolution and speed due to the high inertial permanent magnet. A simple 2 phase permanent magnet type stepper motor is shown in Fig 1.7.

Variable Reluctance Type – VR Type

The VR type stepper motors consist of wire wound stator poles surrounding the rotor poles that are constructed using ferromagnetic materials. The reluctance of magnetic circuit formed by rotor and stator teeth varies with angular position of the rotor and hence it is termed as variable reluctance motor.

The direction of motor rotation is independent of polarity of status current. This construction provides higher speed and resolution as it doesn't have a permanent magnet rotor and instead has a soft iron rotor, but it has less torque and does not hold its position as well compared to other types.

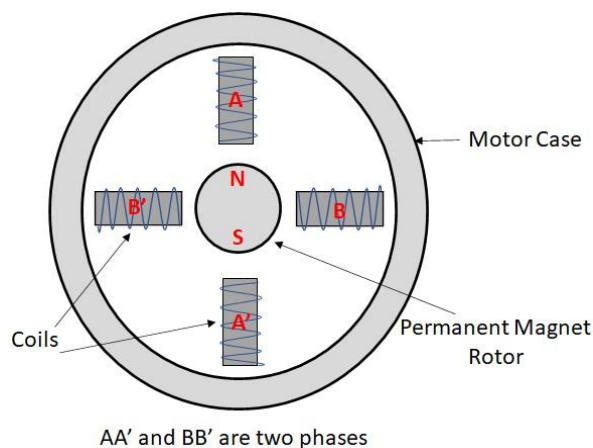


Fig 14.7 Two-Phase Permanent Magnet Type Stepper Motor

Hybrid Type

The hybrid type stepper motor combines the features of the variable reluctance and the permanent magnet feature. Here the rotor is a shaped permanent magnet that results in varying reluctance based on the rotated angle. The Hybrid type stepper motors consist of wire wound stator poles surrounding a permanent magnet rotor pole. As it includes the features of permanent magnet and variable reluctance, it combines the advantage of both these like high speed, resolution and torque. The direction of rotation depends on the polarity of applied status current.

14.2.1.2 Operation of a Stepper Motor:

A simple 2 phase permanent magnet type stepper motor is shown in Fig 14.7. The setup consists of a permanent magnet rotor with north and south poles, and the stator consists of 4 coils where two pairs are formed by connecting 2 coils each. This results in two

windings where only one winding is energised at a time. Hence this construction is termed as 2 phase stepper motor.

Excitation of one winding creates an electromagnetic field with a North & South pole. It results in attraction of the rotors magnetic pole towards excited stator winding causing an alignment of rotor to the new magnetic field that results in a step change of 90° . The process of energizing the stators in a specific sequence result in rotation of the stepper motor by fixed 90° step angles for this setup. The operation of the same is shown in Fig 14.8 for 4 steps to make a rotation.

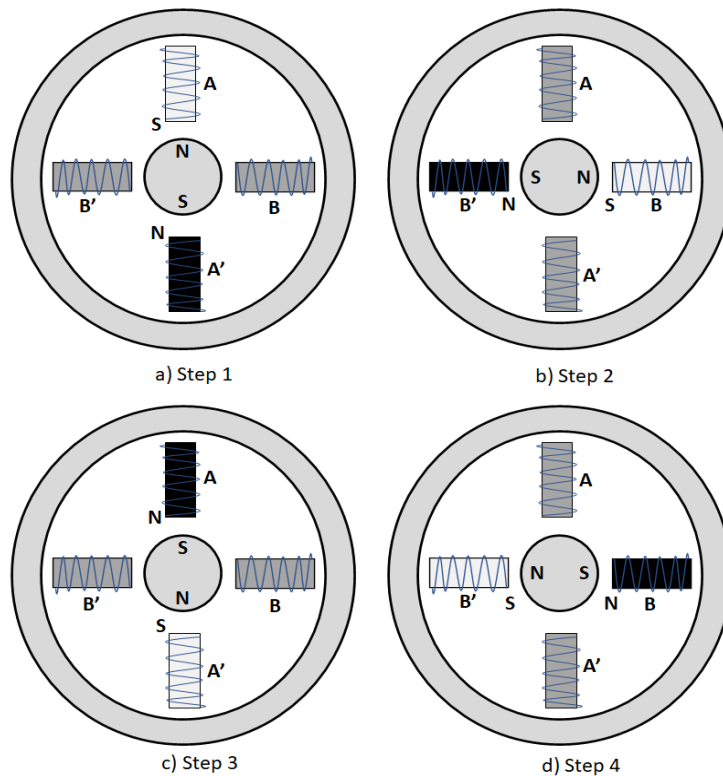
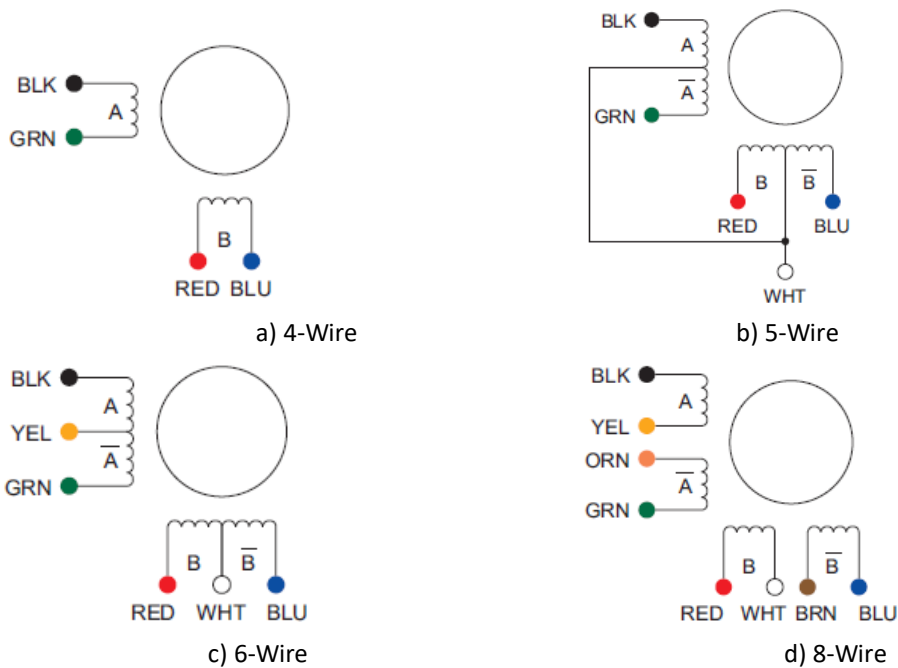


Fig 14.8 Operation sequence steps of Stepper Motor

The step 1 involves the excitation of the winding A creating a south pole and attracting the north pole of rotor towards it and a north pole is created in A' that repels rotors north pole and attracts south pole as shown in fig 14.8a. The step 2 involves the excitation of the winding B creating a south pole and attracting the north pole of rotor towards it and a north pole is created in B' that repels rotors north pole and attracts south pole as shown in fig 14.8b. The step 3 involves the excitation of the winding A' and is similar to step 1 but attracting towards A'. It is shown in fig 14.8c. The step 4 involves the excitation of the winding B' and is similar to step 2 but attracting towards B'. It is shown in fig 14.8d. A

higher resolution is achieved by using higher number of stator poles (results in higher number of phases or coils per phase) and/or rotor magnetic poles/teeth.



4 Lead Bipolar Connection	6 Lead Unipolar Connection	6 Lead Bipolar (Series) Connection
8 Lead Unipolar Connection	8 Lead Bipolar (Series) Connection	8 Lead Bipolar (Parallel) Connection

e) Connection Types

Fig 14.9 Wiring types of Stepper Motors

Source: Oriental Motor

14.2.1.3 Wiring types/Polarity of Stepper Motor:

The most used stepper motors are two phase or four phase motors. They may be classified as follows based on their wiring or polarity. They are

- Universal
- Unipolar
- Bipolar

Universal Type:

The universal wiring type as the name suggests is universal and flexible type which can be wired to produce a Unipolar or a bipolar type. It typically consists of 8 terminals. The different wiring type of stepper motors are shown in Fig 14.9.

Unipolar:

Unipolar stepper motors typically have 2 coils per phase or one winding with centre tap in between. The two phases together have 4 coils with two separate centre taps. The centre taps may be electrically shorted or kept isolated and are typically connected to supply voltage V. It typically contains 6 terminals. It is reduced to 5 terminals when the centre taps of the phases are shorted. This results in supply wire and 4 control wires (A+, A-, B+ and B-) with only half winding used per phase at a given time. Since only half winding is used per phase unipolar stepper motors result in lower torque and efficiency.

A unipolar type simplifies the operation as current flows in only one direction, and it does not require reversal of current in the driving circuit to reverse the direction of the magnetic field. It requires only a simple communication circuit that uses only lower 4 MOSFETS/Diodes for driving the unipolar stepper motor. The current flow is shown in fig 14.10 for Unipolar (a) and Bipolar (b).

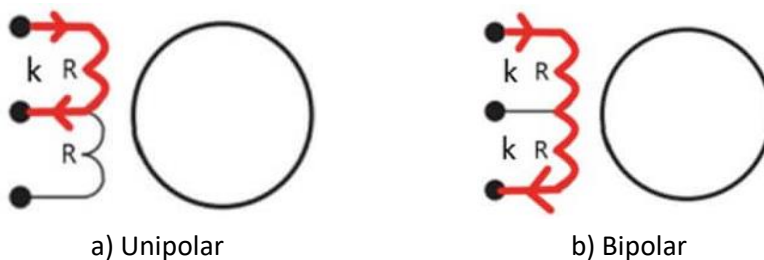


Fig 14.10 Current in Bipolar and Unipolar Motors

Source: Portescap

Bipolar:

A bipolar type utilizes one coil for each phase and has total of 4 terminals. A reversal of current in the winding reverses the electromagnetic field, magnetic poles which reverse

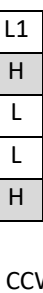
the direction of rotation of the rotor. It is termed as bidirectional as it allows for applying the current in the forward and reverse direction. Hence require complicated driver electronics typically H-Bridge for each phase to reverse the direction of current in winding. Since bipolar types use the full winding, it generates higher torque.

14.2.1.4 Driving Modes – Stepping types of Stepper Motor:

A sequence of excitations is required to produce a rotation in the stepper motor. This sequence of steps is termed as the driving mode of the stepper motor. Various mode types exist and the most common driving modes are

- Wave Step
- Full Step
- Half Step
- Micro Step

Table 14.3 Different stepping sequences for active High Logic

a) Wave Stepping					b) Full Stepping					c) Half Stepping				
Step	L4	L3	L2	L1	Step	L4	L3	L2	L1	Step	L4	L3	L2	L1
1	L	L	L	H	1	L	L	H	H	1	L	L	L	H
2	L	L	H	L	2	L	H	H	L	2	L	L	H	H
3	L	H	L	L	3	H	H	L	L	3	L	L	H	L
4	H	L	L	L	4	H	L	L	H	4	L	H	H	L
 CW					 CCW					5	L	H	L	L
										6	H	H	L	L
										7	H	L	L	L
										8	H	L	L	H

Note:

- Here “H” represents High or logic 1 and “L” represents Low or logic 0. The voltage values for H or L depends on application where H generally represent +5V or +12V and L represent GND or 0V.
- An active high logic requires a “H” or logic “1” to activate/excite a winding and “L” or logic “0” to deactivate. An active low logic requires “L” or logic “0” to activate/excite a winding and “H” or logic “1” to deactivate.

The sequence to be followed for the different types of stepping is shown in Table 14.3. A reverse direction is achieved when the opposite sequence direction is followed i.e. 4, 3, 2, 1 instead of 1, 2, 3, 4.

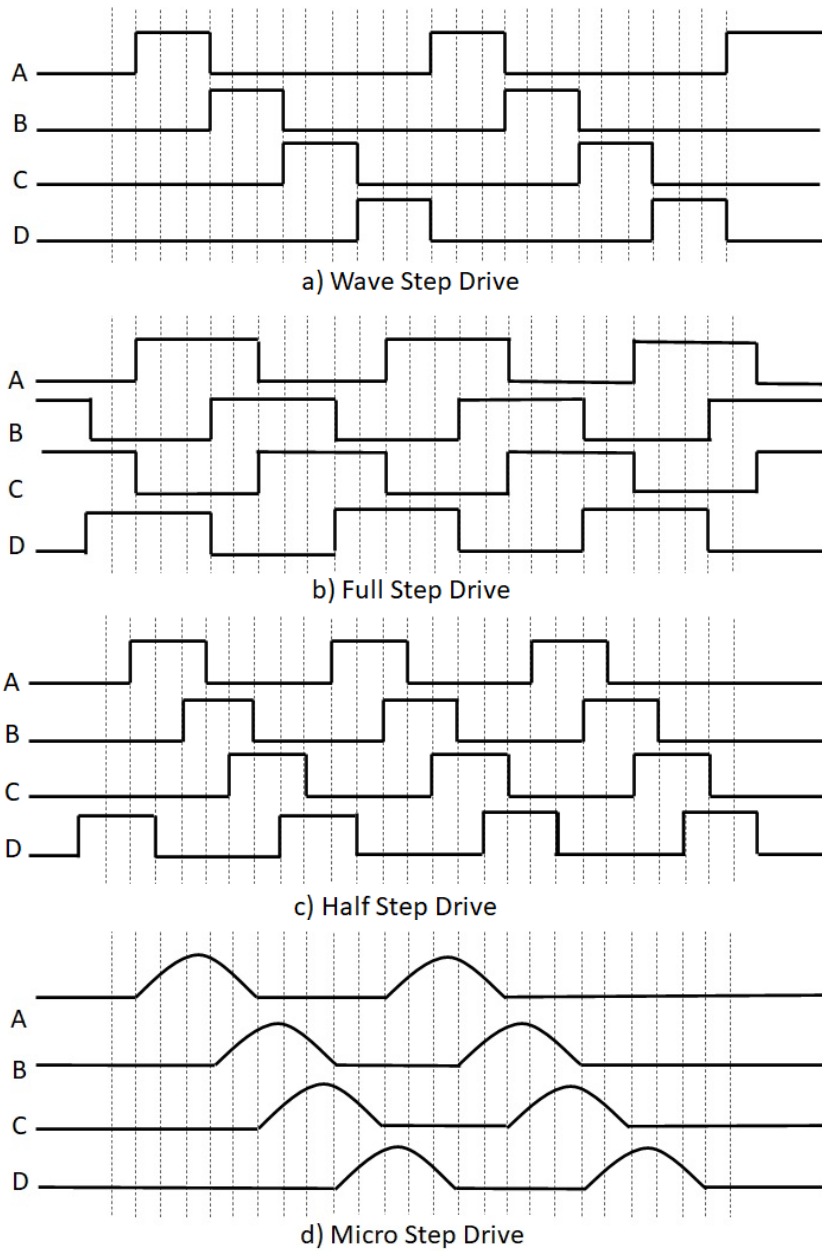


Fig 14.11 Stepping Sequence Waveforms

Wave Stepping:

In this stepping type only one winding is excited at a time as shown in Table 14.3a). The wave stepping provides less torque than full stepping and is the least stable at higher speeds but has low power consumption. It has poor step accuracy and reduced hold/running torque. The waveform is shown in Fig 14.11a.

Full Stepping:

In this stepping type two windings are excited at a time as shown in Table 14.3b). The full stepping provides the lowest resolution and is good at holding its position. It has high torque and good step accuracy. The waveform is shown in Fig 14.11b.

Half Stepping:

The half stepping is a combination of full stepping and wave stepping. In this stepping type 1 or 2 windings are excited in alternating sequence as shown in Table 14.3c. It provides the most torque, resolution and is the most stable at higher speeds. The waveform is shown in Fig 14.11c.

Micro Stepping: Sine Cosine Microstepping

In Micro stepping, a full step is divided into smaller steps for a smooth rotation of the motor, specifically at lower speeds. A fraction of full step angle can be achieved using the micro stepping technique. This is achieved by using pulse-width modulation (PWM) voltage to control the current flowing through the windings of the stepper motor. The control driver sends two sinusoidal voltage signals that are out of phase by 90° to the motor windings. This results in an increase in current in one winding, and a decrease of current in other winding. Since the current is varying gradually and a sudden change of phase does not occur, it produces a smoother rotation of the stepper motor which minimizes the jerking/oscillation effects that normally occur in step changes. This is because the coil windings are neither fully OFF nor fully ON. It also has a more consistent torque production with respect to full or half stepping. The waveform is shown in Fig 14.11d.

14.2.1.5 Parameters/Specifications

The following are parameters/specifications of stepper motors. These values vary based on the construction and type of stepper motors. The definitions and typical values are presented below.

Stepping Angle

The stepper motor rotates in discrete angular steps. The step angle is defined as the angle that the rotor of a stepper motor moves when an excitation is provided to stator winding of a stepper motor. It is expressed in degrees or radians. Various stepper motors are available with step angles varying from 0.72° to 90° . A lower step angle is required for accurate positioning which implies a higher resolution, and it depends on the total number of magnetic poles of the motor. The number of steps for a revolution is given by $360^\circ/\text{stepping angle}$.

Residual Torque

It is the detent torque of permanent magnet stepper motor when it is not energised. It is the result of permanent magnetic flux, inertial and other effects. It is the torque produced on the shaft to maintain the static position when the motor is not excited.

Holding Torque:

It is the amount of torque that is required to move the rotor of the stepper motor by a full step from a stand still position when it is energised. This is typically higher than the running torque. It acts as a brake when the stepper motor is holding a load. Ex: 3kg-cm etc

Running Torque:

It is the torque required to move the rotor of the stepper motor while it is energised. It is typically lower than the Holding torque.

Motor Speed:

The motor speed could be controlled by varying the applied current or to certain extent through the switching speed of winding excitation sequence. For a smooth movement of the rotor, the next winding in the sequence must be excited such a way that the movement is carried forward before the rotor comes to a standstill. If the time delay used between the excitations is large, it causes a distinct stop in movement of the rotor or results in clear discrete step movements.

Voltage, Current & Other ratings:

The typical operating voltages are in the range of 3V to 24V with current in the range of 0.1A to 2A. Other details include the number of available terminals between 4 and 8, size, weight and coil resistance typically around 1.5 to 25 Ohms etc.

14.2.2 Interfacing of Stepper Motors

Stepper Motors could be interfaced using different ways using Transistors, FET's or readily available drivers such as ULN2803C. The ULN2803C driver consists of 8 NPN Darlington Transistor pairs with common-cathode clamp diodes for protection against inductive loads, has a 500mA collector current rating and capable of up to 50V with common emitter and open collector outputs.

The ULN2803C has inputs at pins 1 to 8 denoted by 1B to 8B and outputs to be connected to motors available at pins 20 to 13 denoted 1C to 8C. The Pin 9 is used as a GND and Pin 12 for power supply. For inductive loads, the COM pin must be tied to coil supply voltage, if connected to resistive loads the COM pin may be connected to a supply voltage or left connected.

It also has a 2.7K Ω series resistor and allows for a direction connection with TTL or 5V CMOS devices. A typical interfacing circuit of PIC18 interfaced to a Unipolar Stepper motor using ULN2803C is presented in Fig 14.12.

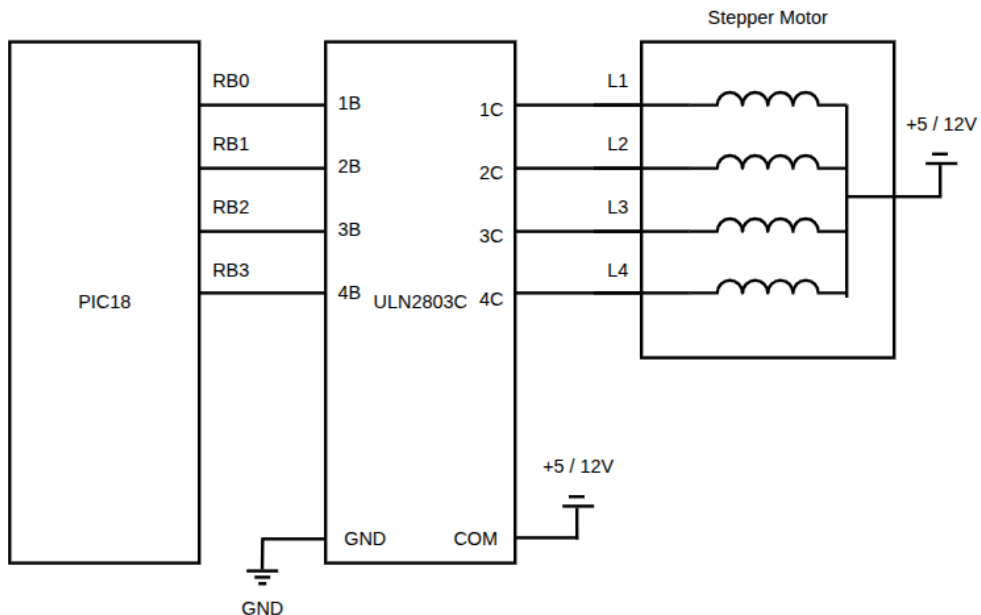


Fig 14.12 PIC18 Interfacing to a Unipolar Stepper motor using ULN2803C

Motor Winding Connections, appropriate loading/exciting value & excitation sequence:

The setup assumes the following. The 4 windings L1 to L4 of stepper motor are connected to lower 4 bits of PORTB bit 0, bit1, bit2 and bit3 respectively as represented in “x x x x L4 L3 L2 L1” of PORTB where x represents pin/bits not used. Observe that the winding L1 is connected to pin 0 of PORTB and so on with L4 connected to pin 3 of PORTB.

In wave stepping, only one winding is excited and it is presented in case a. In full stepping, two windings are excited and it is presented in case b.

a) Wave Stepping:

The excitation sequence for the wave stepping requires a starting value of

- 0x01 with left rotation (using RLNCF instruction) to create a clockwise rotation. But this requires the register/PORTB to be reloaded with the value of 0x01 for every 4 excitations. It is presented here in Program 14.7.
 - This can be overcome when a value of 0x11 is loaded instead of 0x01. It is presented in Program 14.8.
- 0x08 with right rotation (using RRNCF instruction) to create a counter clockwise rotation. But this requires the register/PORTB to be reloaded with the value of 0x08 for every 4 excitations.

- This can be overcome when a value of 0x88 is loaded instead of 0x08. It is presented in Program 14.9.

b) Full Stepping:

The excitation sequence for the full stepping requires a starting value of

- 0x03 with left rotation (using RLNCF instruction) to create a clockwise rotation. But this requires the register/PORTB to be reloaded with the value of 0x03 for every 4 excitations.
 - This can be overcome when a value of 0x33 is loaded instead of 0x03. It is presented in Program 14.10.
- 0x0C with right rotation (using RRNCF instruction) to create a counter clockwise rotation. But this requires the register/PORTB to be reloaded with the value of 0x0C for every 4 excitations.
 - This can be overcome when a value of 0xCC is loaded instead of 0x0C.

Note: It must be noted that the instruction which must be used i.e. (RRNCF or RLNCF) depends on the connecting order of windings. Traditionally for a clockwise winding number 1 to 4 connected to bits<0:3> of PORT requires left shift and RLNCF instruction for clockwise direction of rotation and RRNCF instruction for counter clockwise direction. For connection of windings number 1 to 4 connected to bits <3:0> of PORT requires right shift and RRNCF for clockwise direction and RLNCF for counter clockwise direction.

Program 14.7: Write an ALP to rotate stepper motor by 1 full revolution. Assume a step size of 1.8° . Assume the 4 windings of stepper motor are connected to PORTB<0:3> and operated based on wave stepping.

Sol: Calculation for number of steps: $360^\circ/1.8^\circ = 200 = C8H = 0xC8$.

So, the number of steps is found to be 0xC8

	#INCLUDE <P18F458.INC>		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x50	; Set Origin 0x0050 of DLY Module
DLY	MOVLW	0x03	; WREG = 0x03, the required loop value ; depends on the motor & the oscillator ; frequency used.
	MOVWF	0x11,0	; [0x11] = WREG = 0x03
L2:	MOVLW	0x04	; WREG = 0x04, the required loop value ; depends on the motor & the oscillator ; frequency used.
	MOVWF	0x10,0	; [0x10] = WREG = 0x04
L1:	DECf	0x10, F	; Decrement Register 0x10, [0x10] = [0x10] - 1

	BNZ	L1	; If not zero Branch to L1.
	DECF	0x11, F	; Decrement Register 0x11, [0x11] = [0x11] - 1
	BNZ	L2	; If not zero Branch to L2.
	RETURN		; Return from Delay module
	ORG	0x0100	; Set Origin 0x0100 of Main Program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISB	; Configure PORTB as output. TRISB = 0x00
	MOVLW	0x32	; Steps for 1 Rotation = 0xC8 = 200d = 360/1.8 ; Loop Count when using Re-initialization for ; every 4 excitations → WREG = 0x32 = 50d = ; 200d /4
	MOVWF	0x21,0	; File Reg [0x21] = WREG = 0x32
	MOVLW	0x01	; WREG = 0x01
	MOVWF	0x20,0	; File Reg [0x20] = 0x01
	MOVWF	0x30,0	; File Reg [0x30] = [0x20] = 0x01
M2:	MOVLW	0x04	; WREG = 0x04
	MOVWF	0x22,0	; File Reg [0x22] = 0x04
	MOVFF	0x30,0x20	; Copy File Reg 0x30 to File Reg 0x20, i.e ; restore 0x01 into 0x20, otherwise the no ; winding gets excited as 1 is shifted to higher ; nibbles 4-bits after 4 rotations.
M1:	MOVF	0x20,0,0	; Move content of file reg 0x20 to WREG, ; WREG = [0x20],
	MOVWF	PORTB,0	; PORTB = WREG
	CALL	DLY	; Call Delay module.; Since MC operates in ; Micro Seconds, to give sufficient time for the ; motor to actually, make required step change. ; The required delay value depends on the ; motor & the oscillator frequency used.
	RLNCF	0x20, F,0	; Rotate Left without Carry, ; [0x20] = [0x20]<<1, LSB = MSB
	DECF	0x22, F,0	; Decrement File Reg 0x21 content by 1, ; [0x21] = [0x21]-1
	BNZ	M1	; Repeat for 4 times for 4 excitations. ; Branch if not zero i.e. if [0x22] content is not ; 0. If 0 go to next
	DECF	0x21, F,0	; Decrement File Reg 0x21 content by 1, ; [0x21] = [0x21]-1
	BNZ	M2	; Repeat for 50 times, total 200 excitations to ; make 1 rotation. Branch if not zero i.e. if [0x10] content is not 0; If 0 go to END
	END		

Program 14.8: Write an ALP to rotate stepper motor by 270° in clockwise direction. Assume a step size of 1.8° . Assume the 4 windings of stepper motor are connected to PORTB<0: 3> and operated based on wave stepping.

Sol: Calculation for number of steps: $270^\circ/1.8^\circ = 150 = 96H = 0x96$. So, the number of steps is found to be 0x96. Consider 0x11 is loaded instead of 0x01 and assuming the 4 windings of stepper motor are connected at xxxxL4L3L2L1, program for a clockwise rotation is given as follows.

	#INCLUDE <P18F458.INC>		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start the Main program.
	ORG	0x50	; Set Origin 0x0050 of DLY Module
DLY	MOVLW	0x03	; WREG = 0x03, the required loop value ; depends on the motor & the oscillator ; frequency used.
	MOVWF	0x11,0	; [0x11] = WREG = 0x03
L2:	MOVLW	0x04	; WREG = 0x04, The required loop value ; depends on the motor & the oscillator ; frequency used.
	MOVWF	0x10,0	; [0x10] = WREG = 0x04
L1:	DECf	0x10, F	; Decrement Register 0x10, [0x10] = [0x10] - 1
	BNZ	L1	; If not zero Branch to L1.
	DECf	0x11, F	; Decrement Register 0x11, [0x11] = [0x11] - 1
	BNZ	L2	; If not zero Branch to L2.
	RETURN		; Return from Delay module
	ORG	0x100	; Set Origin 0x0100 of Main Program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISB	; Configure PORTB as output. TRISB = 0x00
	MOVLW	0x96	; WREG = 0x96= 150d = 270/1.8
	MOVWF	0x21,0	; File Reg [0x21] = WREG = 0x96
	MOVLW	0x11	; WREG = 0x11
	MOVWF	0x20,0	; File Reg [0x20] = 0x11
M1:	MOVf	0x20,0,0	; Move content of file reg 0x20 to WREG, ; WREG = [0x20]
	MOVWF	PORTB,0	; PORTB = WREG
	CALL	DLY	; Call Delay module. ; Since MC operates in ; Micro Seconds, to give sufficient time for the ; motor to actually make required step change. ; The required delay value depends on the ; motor & the oscillator frequency used.
	RLNCF	0x20, F,0	; Rotate Left without Carry, ; [0x20] = [0x20] <<1, LSB = MSB

	DECF	0x21, F,0	; Decrement File Reg 0x21 content by 1, ; [0x21] = [0x21] - 1
	BNZ	M1	; Repeat for 150d times to make 270 Degree ; rotation. Branch if not zero i.e. if [0x21] ; content is not 0. If 0 go to END
	END		

Program 14.9: Write an ALP to rotate stepper motor by 360 degrees revolutions anti clockwise to open a garage door. After a delay of 10 seconds, close the garage door by rotating clockwise up to 360 degrees. Assume a step size of 1.8° and that the 4 windings of stepper motor are connected to PORTB<0:3>, operated based on wave stepping.

Sol: Calculation for number of steps for 360° :

$360^\circ / 1.8^\circ = 200 = 0xC8$. So, the number of steps is found to be 0xC8.

For clockwise direction RLNCF instruction is used with initial value of 0x11, whereas for anticlockwise direction RRNCF instruction is used with initial value of 0x88. The program is given as follows.

	#INCLUDE <P18F458.INC>		
	ORG	0x00	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x50	; Set Origin 0x0050 of DLY Module
DLY	MOVLW	0x03	; WREG = 0x03, The required loop value ; depends on the motor & the oscillator ; frequency used.
	MOVWF	0x11,0	; [0x11] = WREG = 0x03
L2:	MOVLW	0x04	; WREG = 0x04, The required loop value ; depends on the motor & the oscillator ; frequency used.
	MOVWF	0x10,0	; [0x10] = WREG = 0x04
L1:	DECF	0x10, F	; Decrement Register 0x10, [0x10] = [0x10] - 1
	BNZ	L1	; If not zero Branch to L1.
	DECF	0x11, F	; Decrement Register 0x11, [0x11] = [0x11] - 1
	BNZ	L2	; If not zero Branch to L2.
	RETURN		; Return from Delay module
	ORG	0x100	; Set Origin 0x0100 of RTN Module
RTN	NOP		; No Operation
M1:	MOVF	0x20,0,0	; Move content of File Reg 0x20 to WREG, ; WREG = [0x20]
	MOVWF	PORTB,0	; PORTB = WREG
	CALL	DLY	; Call Delay module; Since MC operates in ; Micro Seconds, to give sufficient time for the ; motor to actually make required step change.

			; The required delay value depends on the ; motor & the oscillator frequency used.
	BTFSS	0x22,0,0	; Test Bit 0 in file reg 0x22.
	BRA	T1	; Branch to T1 if 0
	RRNCF	0x20, F,0	; Rotate Right without Carry, ; [0x20] = [0x20] <<1, LSB = MSB
	BRA	T2	; Branch to T2
T1:	RLNCF	0x20, F,0	; Rotate Left without Carry, ; [0x20] = [0x20] <<1, LSB = MSB
T2:	DECF	0x21, F,0	; Decrement File Reg 0x21 content by 1, ; [0x21] = [0x21] - 1
	BNZ	M1	; Repeat for 150d times to make 270 Degree ; rotation. Branch if not zero i.e. if [0x21] content is ; not 0. If 0 go to END
	RETURN		
	ORG	0x200	; Set Origin 0x0200 of Main Program
START	MOVLW	B'00000000'	; WREG = 0x00
	MOVWF	TRISB	; Configure PORTB as output. TRISB = 0x00
	MOVLW	0x11	; WREG = 0x11
	MOVWF	0x20,0	; File Reg [0x20] = 0x11
	MOVLW	0xC8	; WREG = 0xC8 = 200d = 360/1.8
	MOVWF	0x21,0	; File Reg [0x21] = WREG = 0xC8
	BCF	0x22,0,0	; Clear Bit 0 of File Reg [0x22] to 0
	CALL	RTN	; Call RTN Module
	CALL	TMDLY	; Call Timer Delay module for 10 Seconds ; Not presented as part of this program. ; Concept discussed in Timer Section of Ch.9; ; Or use equivalent DLY subroutine given here.
	MOVLW	0x88	; WREG = 0x88
	MOVWF	0x20,0	; File Reg [0x20] = 0x88
	MOVLW	0xC8	; WREG = 0xC8 = 200d = 360/1.8
	MOVWF	0x21,0	; File Reg [0x21] = WREG = 0xC8
	BSF	0x22,0,0	; Set Bit 0 of File Reg [0x22] to 1
	CALL	RTN	; Call RTN Module
	END		

Program 14.10: Write an ALP to rotate stepper motor by 270° in clockwise direction. Assume a step size of 1.8° . Assume the 4 windings of stepper motor are connected to PORTB<0:3> and operated based on full stepping.

Sol: The program is identical to Program 14.x2 with the exception that a 0x33 is used instead of a 0x11. This ensures that two windings are excited at a time and provides full stepping.

The “MOVLW 0x33 ; WREG = 0x33” instruction must be used instead of “MOVLW 0x11 ; WREG = 0x11”. It is as shown below with no highlighting/background colour, and which is the 5th instruction from the START label in Program 14.x2.

	MOVLW	0x33	; WREG = 0x33
	MOVWF	0x20,0	; File Reg [0x20] = 0x33

14.3 Interfacing of Servo Motors

Stepper motors are used for positional change or movement in steps. However, some applications require smooth motion and high-precision control of shaft position and speed. In such cases, a feedback and control mechanism is employed to achieve accurate positioning and motion control. Servo motors are designed with feedback and control mechanism for applications involving either rotational or linear (translatory) motion. The servo motors offer high torque, efficiency, low cost and complexity in addition to precision control compared to a stepper motor. Servo motors are controlled by analog or digital signals using Servo Driver, Microcontroller & PWM signals. The PWM duration and frequency of the pulses determine the position of the shaft. The generation of PWM signals is presented in Ch 13 using CCP and ECCP.

Various types of servo motors are available from different manufacturers. They are mainly categorized as follows:

- AC Servo Motors
- DC Servo Motors
- Brushed Servo Motors
- Brushless Servo Motors
- Synchronous Motors
- Asynchronous Motors
- Linear Servo Motors

Readers are encouraged to refer details of manufacturers such as Advanced Motion Controls etc. for more information.